



DOCUMENTACION TECNICA · TOTAL CLEAN CAR

Manual de administrador — Total Clean Car

Proyecto: Total Clean Car

Version: 2.0

Fecha: 27/09/2026

Elaborado por: Auditoria tecnica

Versión 2.0 · 2026-09-27 · Reemplaza a la versión 1.0 (2026-09-15, "LavadoCarros")

Manual técnico: roles, catálogo, reglas de agenda, parámetros, despliegue y mantenimiento. El uso de las pantallas está en `07_MANUAL_USUARIO/Manual_Usuario.md` (secciones 6 y 7). Los endpoints completos, en Swagger: `http://127.0.0.1:3042/api/docs`.

1. Usuarios y roles

Cuatro roles: `admin`, `operador`, `lavador`, `cliente`. Distribución al 2026-09-27: 7 clientes, 3 lavadores, 1 operador, 1 admin.

Rol	Alcance	Inicio al entrar
<code>admin</code>	administración completa	<code>/dashboard</code> (en el celular o la APK, <code>/agenda</code>)
<code>operador</code>	agenda, reservas, asignaciones, validación de pagos, catálogo	<code>/dashboard</code> (en el celular o la APK, <code>/agenda</code>)
<code>lavador</code>	sus trabajos y carga de evidencia	<code>/mis-trabajos</code>
<code>cliente</code>	sus reservas, vehículos, pagos y calificaciones	<code>/mis-reservas</code>

Suspender una cuenta corta el acceso de inmediato. El sistema revalida el estado en cada petición, así que no hay que esperar a que caduque el token. Basta poner `estado` distinto de `activo` en `Usuario`, o en la ficha de `Cliente` o `Lavador`.

Advertencia: el rol es texto libre en la base. Al crear usuarios **por fuera de la aplicación**, un valor mal escrito (`"Admin"` en vez de `"admin"`) deja la cuenta sin permisos, sin mensaje de error. Usar siempre minúsculas y exactamente uno de los cuatro valores.

1.1 Credenciales

Estas tres acciones son **solo del** `admin`:

Acción	Endpoint
Crear el acceso o restablecer la contraseña de un cliente	<code>POST /api/clientes/:id/acceso</code>
Crear o editar la cuenta de acceso de un lavador	<code>PUT /api/lavadores/:id/usuario</code>
Restablecer la contraseña de un lavador	<code>POST /api/lavadores/:id/usuario/reset</code>

El sistema genera una contraseña temporal que se muestra **una sola vez**.

Pendiente de seguridad (2026-09-27): `admin`, `operador`, `lavador1` y `lavador2` siguen con las contraseñas iniciales de `backend/prisma/seed.js`, que está publicado en GitHub. Cambiarlas antes de dar acceso a más personas o de publicar la app con dominio. Los clientes `*@demo.com` son de demostración.

2. Catálogos

Catálogo	Entidad	Registros al 2026-09-27
Servicios	<code>TipoServicio</code>	6
Tipos de vehículo	<code>TipoVehiculo</code>	4 (moto, liviano, SUV, camioneta)
Precios por tipo	<code>PrecioServicio</code>	18 activos
Servicios adicionales	<code>ServicioAdicional</code>	0
Estacionamientos	<code>Estacionamiento</code>	5
Plazas y bahías	<code>Plaza</code>	42
Planes	<code>Plan</code>	1 (2 suscripciones activas)

Ningún registro de catálogo se borra: se desactiva (`activo = false` o `estado = 'inactivo'`). Las reservas guardan su propia copia de lo que se cobró. El catálogo lo pueden editar `admin` y `operador` .

3. Precios por tipo de vehículo

El precio y la duración **no** están en `TipoServicio` : están en `PrecioServicio` , una fila por servicio y tipo de vehículo (única por `id_tipo_servicio + id_tipo_vehiculo`).

- Un servicio se ofrece a un tipo solo si existe esa fila, con `activo = true` , y el tipo está activo.
- La duración de la fila define cuánto se bloquea en la agenda.
- `GET /api/public/servicios` devuelve cada servicio con sus precios y `precio_desde` ; omite los que no tienen ningún precio.
- `POST` y `PUT /api/servicios` reciben `precios: [{ id_tipo_vehiculo, precio, duracion_min }]` . Un tipo que deja de venir queda desactivado, no borrado. No se acepta un servicio sin ningún precio.
- Tipos de vehículo: `GET/POST/PUT /api/tipos-vehiculo` (interno) y `GET /api/public/tipos-vehiculo` . El `codigo` se genera a partir del nombre.

`Vehiculo.id_tipo_vehiculo` es obligatorio al registrar (`POST /api/vehiculos`).

Admite `null` solo por los vehículos anteriores al catálogo: **3 vehículos** siguen sin tipo y no pueden reservar hasta que su dueño o el operador lo complete. El cliente puede editar sus propios vehículos (`PUT /api/vehiculos/:id`), pero no su estado.

Pendiente (2026-09-27): el tipo **moto** no tiene ningún precio, así que ningún servicio se le ofrece. La migración no inventó tarifas: se cargan desde **Servicios → Servicios y precios**.

4. Servicios adicionales

`ServicioAdicional` (nombre, precio, `duracion_min` extra). CRUD interno en `/api/adicionales`; lectura pública en `/api/public/adicionales`.

Al reservar, cada adicional elegido se copia a `ReservaAdicional` (nombre, precio y minutos): cambiar el catálogo después no altera reservas ya hechas.

`Reserva.precio_final` = precio del servicio para el tipo de vehículo (0 si lo cubre un plan) + adicionales. **Los planes no cubren adicionales.**

5. Cotización y creación de reservas

El precio y la duración los calcula siempre el backend (`services/cotizacion.service.js`); lo que muestra la pantalla sale de la misma cuenta.

- `POST /api/reservas/cotizar` — vista previa: `{ id_vehiculo, id_tipo_servicio, adicionales }`.
- `POST /api/reservas` — crea la reserva, con `adicionales: [ids]` opcional.

Validaciones al crear, en este orden: cliente activo, vehículo del cliente y activo, servicio ofrecido para el tipo de vehículo, estacionamiento (en expreso), fecha no pasada, **hora no empezada** (si es hoy), dentro del horario de atención del sitio, y cupo disponible. El cupo se verifica en una transacción

serializable: si dos clientes piden el último cupo a la vez, uno lo obtiene y el otro recibe `409`.

6. Agenda y capacidad

Modalidad	Capacidad por franja
Expreso	<code>Estacionamiento.capacidad_expreso</code> del sitio; si es 0, la cantidad de lavadores activos
Profunda	<code>bahías (Plaza.tipo = 'bahia_lavado')</code> disponibles

- **Horarios ofrecidos:** empiezan en la apertura del sitio y avanzan cada `duracion_franja_min` (60 por defecto) para servicios que duran eso o más; los más cortos, cada 30 minutos. Así los horarios caen en :00 y :30.
- **Franjas pasadas:** hoy, las que ya empezaron salen con `pasada: true`, sin cupos, y no se pueden reservar.
- **La disponibilidad y la creación usan la misma regla** (`services/agenda.service.js`). Hasta el 2026-09-27, crear contaba la capacidad global y un sitio con capacidad propia podía aceptar reservas encimadas.

- **Bloqueos manuales:** `POST /api/agenda/bloqueos` y `DELETE /api/agenda/bloqueos/:id`. Bloquean ambas modalidades en todos los sitios.
- **Asignar lavador:** `PUT /api/reservas/:id/asignar-lavador`. Rechaza con `409` si el lavador ya tiene otro trabajo solapado ese día.
- **Estados:** `solicitada` → `confirmada` → `en_proceso` → `completada`, con salidas a `cancelada` (desde `solicitada` o `confirmada`) y `no_asistio` (desde `confirmada` o en proceso). `PUT /api/reservas/:id/no-asistio` existe, pero **el panel no tiene botón** para marcarlo.

7. Fechas y zona horaria

`Reserva.fecha` es un día, guardado como **medianoche UTC** (`2026-09-27T00:00:00Z`). El contenedor corre en UTC; "hoy", "ahora", los KPIs del día y los recordatorios se calculan en `APP_TZ` (por defecto `America/Guayaquil`). Todo pasa por `backend/src/utils/fechas.js` y `frontend/src/utils/fechas.ts`.

Antes del 2026-09-27 el navegador (UTC−5) mostraba cada reserva un día antes, y desde las 19:00 el servidor ya contaba el día siguiente. No volver a usar `new Date(fecha)` con `setHours`, ni `toLocaleDateString()` sobre `Reserva.fecha`.

8. Validación de pagos

Los comprobantes que sube el cliente quedan `en_verificacion` y no cuentan como ingreso. Se listan con `GET /api/pagos/pendientes` y se resuelven con `PUT /api/reservas/:id/pagos/:pagoId/verificar`. Un comprobante es un archivo de hasta 5 MB (JPG, PNG, WEBP, GIF o PDF).

La **pasarela de tarjeta no está conectada**: `POST /api/reservas/:id/pagos/tarjeta` deja el pago pendiente, y la pantalla del cliente no ofrece tarjeta. Antes de conectarla, resolver SEC-02: el webhook no valida firma.

9. Reportes

Reporte	Endpoint
KPIs	GET /api/reportes/kpis
Ingresos	GET /api/reportes/ingresos
Por estacionamiento	GET /api/reportes/por-estacionamiento
Por servicio	GET /api/reportes/por-servicio
Exportar Excel	GET /api/reportes/exportar/excel
Exportar PDF	GET /api/reportes/exportar/pdf
Acta de servicio	GET /api/reportes/reserva/:id/acta.pdf

El acta incluye el tipo de vehículo y los adicionales con su precio. Los PDF llevan el logo desde `backend/assets/logo-mark.png`.

10. Parámetros del sistema

Todo en `.env`, junto a `docker-compose.yml`. **Es de `server-dt` con permisos `600`**: no volver a dejarlo legible por todos (hasta el 2026-09-27 era `644` de root).
Un cambio requiere recrear el contenedor: `docker compose up -d backend`.

Variable	Efecto
<code>JWT_EXPIRES_IN</code>	duración de la sesión (hoy <code>24h</code>)
<code>APP_TZ</code>	zona horaria del negocio (por defecto <code>America/Guayaquil</code>)
<code>CORS_ORIGINS</code>	orígenes permitidos, separados por coma; vacío = abierto. La web no lo necesita; la APK usa el origen <code>http://localhost</code>
<code>RATE_LIMIT_GENERAL</code>	límite de peticiones general
<code>RATE_LIMIT_OFF</code>	desactiva los límites
<code>SMTP_*</code>	envío de correo; el remitente se muestra como "Total Clean Car"
<code>TWILIO_*</code>	WhatsApp y SMS

`RATE_LIMIT_OFF` desactiva la protección contra fuerza bruta. Usarlo solo en pruebas y nunca dejarlo activo en producción.

Los recordatorios salen todos los días a las **18:00 hora de `APP_TZ`**, para las reservas del día siguiente.

11. Despliegue y migraciones

El esquema **no** usa `prisma migrate` (no hay tabla `_prisma_migrations`): se aplicó con `db push`. Los cambios de esquema son SQL a mano en `backend/prisma/migraciones/`, cada uno dentro de una transacción.

Migración	Aplicada en producción
2026-09-27_tipos_vehiculo_y_adicionales.sql	2026-09-27 12:26

Procedimiento de despliegue (el usado el 2026-09-27, con un corte de unos 30 s):

```
cd /home/server-dt/Documentos/lavado_de_carros
# 1. Respaldo de base, archivos subidos y .env (ver sección 13)
# 2. Compilar las imágenes con el servicio aún en marcha
docker compose build backend frontend
# 3. Solo si hay migración: detener el backend y aplicarla
docker compose stop backend
docker exec -i lavado_de_carros-postgres-1 psql -U lavado_user -d lavado_db \
  -v ON_ERROR_STOP=1 < backend/prisma/migraciones/<archivo>.sql
# 4. Levantar y verificar
docker compose up -d backend frontend
curl -s http://127.0.0.1:3041/api/health
docker exec lavado_de_carros-backend-1 sh -c \
  'npx prisma migrate diff --from-url "$DATABASE_URL" --to-schema-datamodel
  prisma/schema.prisma --exit-code'
# debe responder "No difference detected"
```

Para probar una migración antes de aplicarla: `pg_dump -Fc` de producción, `pg_restore` en un Postgres temporal y correr el SQL ahí.

Commits y push como `server-dt`, no como root (`sudo -u server-dt -H git ...`): el repositorio tiene identidad local `megaman0012` y su remoto usa la clave personal.

12. App Android

Ver `15_APK/Preparacion_APK.md`. Estado al 2026-09-27: APK de **prueba** (`/home/server-dt/apk/TotalCleanCar-1.0-prueba.apk`), firmada con la clave de depuración, contra `http://181.188.232.50:3041` **sin cifrar**. Para publicarla en Google Play faltan un dominio con HTTPS y un keystore propio.

La APK trae su propia copia del frontend: **un cambio en la web no llega a la app hasta recompilarla** y reinstalarla.

13. Respaldos y recuperación

Ver `12_CONTINUIDAD/Backup_Recuperacion.md`. **Sigue sin haber respaldo automático** (CONT-01).

Existe un respaldo manual, previo al cambio a Total Clean Car:

```
/home/server-dt/respaldos/lavado_de_carros/2026-09-27_pre-total-clean-car/  
lavado_db.dump      pg_dump -Fc de la base  
uploads_data.tgz    evidencias y comprobantes  
env.backup          copia del .env (permisos 600)  
git_head.txt        commit desplegado en ese momento
```

Para volver a ese estado: restaurar `lavado_db.dump` con `pg_restore --clean`, volver el código al commit de `git_head.txt` y reconstruir los contenedores.

14. Mantenimiento

- **Base de datos:** PostgreSQL 15 con autovacuum por defecto. Sin tareas manuales para este volumen.
- **Archivos subidos:** el volumen `uploads_data` (884 KB al 2026-09-27) **crece sin límite**. No hay política de purga; conviene definir una retención antes de que escale.
- **Caché de la web:** el service worker pide `index.html` a la red primero y nginx lo sirve con `no-cache`, así que una versión nueva llega al recargar. Si se cambia `public/service-worker.js`, subir su `CACHE_NAME`.

15. Monitoreo y logs

```
docker compose ps  
curl -s http://127.0.0.1:3042/api/health  
docker compose logs -f backend
```

Sin monitoreo automático. El backend y el frontend no tienen healthcheck; solo PostgreSQL lo tiene.